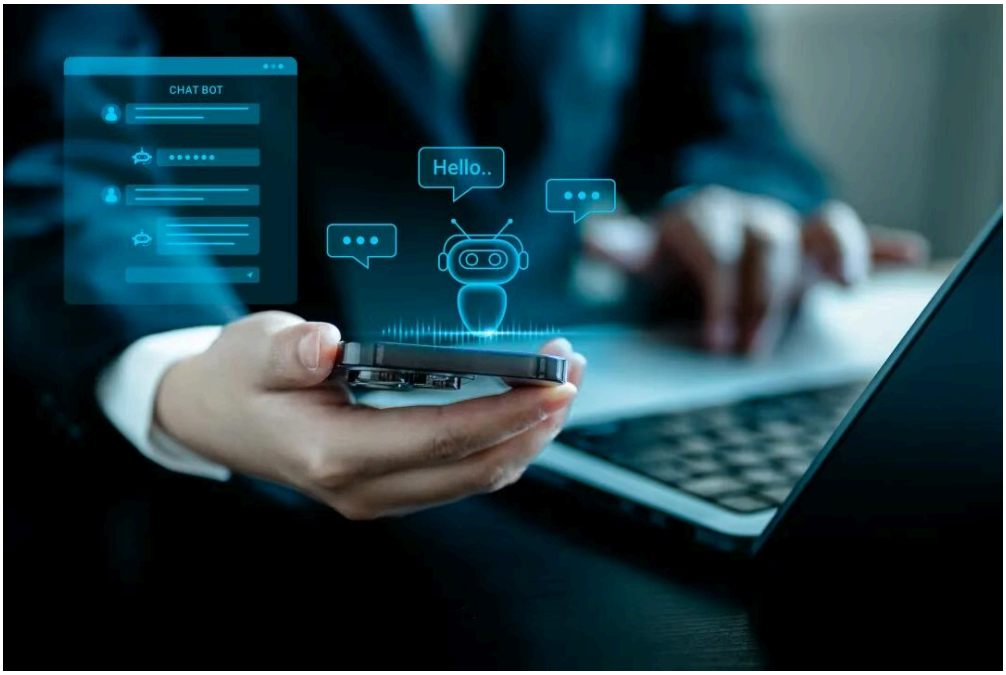
Harness 工程之道：Skill 原理与最佳实践

张梦杰 阿里技术 2026年8月17日 04:05 浙江 107人



这是2026年的第 52 篇文章

（ 本文阅读时间：约 15 分钟 ）

Agent Skills 是一种轻量、开放的能力扩展规范，用于为 AI Agent 扩展专业知识和工作流。本文从概念原理出发，结合真实的工程化项目 trade-ab-skill，系统性地讲解 Skill 的结构规范、触发机制、作用域优先级，以及最佳实践。（文章内容基于作者个人技术实践与独立思考，旨在分享经验，仅代表个人观点。）

01
Skill 的产生与核心理念

1.1 从 Prompt 工程到 Skill 工程的演进

当前的大模型都存在一个共性：**每轮对话都是「失忆」的，必须从零开始构建上下文**，由此发展出了 Prompt 工程。比如 CLAUDE.md 将团队的技术选型、编码规范、架构约束等都固化为一份 Agent 可读的配置文档。每次会话启动时，Agent 就会自动加载这些信息，直接进入高效协作模式，不需要每次都从头到尾交代背景。

但是随着复杂性的增加，问题也逐渐暴露出来。传统的 Prompt 工程将所有领域知识一股脑塞进提示词，项目越复杂，Prompt 越臃肿。**上下文窗口被撑满，模型的注意力被稀释**，真正关键的信息反而容易被"淹没"。并且这些**知识和具体项目深度耦合**，换个场景就得重写一遍，几乎没有复用性可言。

Agent Skill 的出现正是为了解决这些问题，**它将领域知识和工作流封装为可移植、可版本控制的文件夹，用来"教"Agent 如何处理特定任务或工作流，Agent 按需加载**。你可以将 Skills 类比成新员工手册。这也是 Anthropic 官方博客用的类比：Skills 是给 AI Agent 的「入职指南」，把领域知识打包成可发现、模块化的能力。

下面是 Agent Skills 的官方定义：

> "Agent Skills are a lightweight, open format for extending AI agent capabilities with specialized knowledge and workflows."

1.2 核心理念：渐进性披露

Skill 体系最核心的设计理念是**渐进性披露（Progressive Disclosure）**，即只在需要时才加载需要的知识，而非一次性加载全部。

Skill 的渐进性披露主要分三阶段：

- **Discovery（发现）阶段**：当会话启动时仅会加载每个 Skill 的 name 和 description，这部分是以常驻方式注入 Agent 上下文中的。
- **Activation（激活）阶段**：当任务匹配上 description 时，将读取完整的 SKILL.md，加载 SKILL.md 中的路由表和全局规则，但还不加载子模块。
- **Execution（执行）阶段**：按路由表加载对应的模块文件，按需读取参考文档，只加载当前任务真正需要的知识。

该模型在实际应用中带来的性能提升是显著的，绝大部分请求仅需要部分资源。渐进性披露用最小的上下文成本，换取了最大的知识覆盖范围，确保上下文窗口留给真正重要的信息，这就是"按需投放知识"的核心经济学优势。同样也保证了决策的精准性和 Skill 的可扩展性。

1.3 System Prompt 与 Skill 的区别

文件类型	System Prompt	Skill
定位	项目级全局规则、编码规范	特定领域能力封装
加载策略	会话启动时全量加载	渐进式按需加载
生效范围	当前项目	可跨项目、跨会话（取决于作用域配置）
上下文成本	恒定占用，与任务无关也会消耗	仅在命中时加载，未命中零成本
结构化	单文件，扁平组织	多文件模块化，支持脚本和资源
适用场景	编码风格、项目约定、通用约束	完整工作流、多步骤流程、领域专家知识

简单记法：System Prompt 是"这个项目的规矩"，Skill 是"一种可复用的能力"。

一个项目可以同时拥有 System Prompt 和多个 Skill：System Prompt 定义全局编码规范和项目约定，Skill 封装特定领域的工作流。两者协同工作——System Prompt 中的规则对所有 Skill 生效，而 Skill 内部的规则仅在激活时叠加。

02
Skill 的结构组成

2.1 目录结构规范

一个 Skill 的核心就是一个包含 `SKILL.md` 的文件夹：

```
my-skill/              # 必需：skill名称，短横线分隔
├── SKILL.md           # 必需：主文件，包括元信息 + 指令，文件名需全大写
├── scripts/           # 可选：可执行脚本
├── references/        # 可选：参考文档
├── assets/            # 可选：模板、资源
└── ...                # 任意额外文件
```

这种格式已被 Claude Code、Cursor、GitHub Copilot、Gemini CLI 等 40+ 主流 Agent 产品

一个设计良好的 Skill 目录结构，是渐进性披露能否有效落地的基础。核心思想是**主文件做路由，模块文件做执行。**

2.2 SKILL.md 文件结构

SKILL.md 是每个 Skill 的唯一入口，也是唯一的"触发器"，其结构分为两部分：**frontmatter 元信息**和**正文指令**。

frontmatter 是 YAML 格式的元信息块，最核心的两个字段是 `name` 和 `description`：

```
---
name: trade-ab-skill
description: 为用户提供 AB 实验的创建与修改能力，支持实验创建、调流量、
  加桶删桶、实验下线等操作。当用户提到创建实验、修改实验、调流量、加桶删桶、
  实验下线等场景时触发。
---
```

下面是以 Claude Code 为例的完整 frontmatter 字段：

维度	字段	是否必填	约束	用途

身份字段（触发机制）	`name`	必填	最长 64 字符，只能用小写字母、数字和连字符，不能以连字符开头或结尾；若省略默认使用目录名	Skill 的唯一标识符，同时也是文件夹名。Agent 通过它来定位和管理 Skill
	`description`	必填	最长 1024 字符，不能为空；若省略默认使用 SKILL.md 正文第一段	Skill 能否被正确触发的关键。Agent 靠这段描述来判断当前任务是否该调用这个 Skill，需要写清楚做什么（WHAT）以及什么时候该用它（WHEN）
	`argument-hint`	可选	参数提示格式，如`[source directory] [output format]`	在`/`菜单中显示，帮助用户了解该 Skill 接受的输入格式
权限字段（权限控制）	`disable-model-invocation`	可选	布尔值（true/false）	设为 `true` 时，禁止模型自动调用该 Skill，仅允许用户手动触发
	`user-invocable`	可选	布尔值（true/false）	设为 `false` 时，用户不能直接调用该 Skill，只有模型可以自动调用
	`allowed-tools`	可选	工具名列表，如 `Read, Grep, Glob, Write, Bash(python:*)`	工具白名单，精确控制 Skill 执行时可调用的工具及其权限范围
	`model`	可选	模型名称，如 `haiku`、`sonnet`	指定该 Skill 使用的模型；建议简单任务用 Haiku 提升响应速度并降低成本
执行字段（运行时环境）	`context`	可选	可选值：`fork`	执行上下文；设为 `fork` 时，将

				在隔离的子智能体中执行，确保不污染主对话上下文
	agent	可选	可选值：Explore、Plan、general-purpose 或自定义 Agent	子智能体类型；当 context 设为 fork 时生效
	hooks	可选	支持 PreToolUse、PostToolUse 等钩子事件	生命周期事件钩子，定义 Skill 激活期间的事件处理逻辑
	version	可选	语义化版本号（semver），如 1.0.0	用于版本控制和追踪迭代，方便团队协作时知道当前用的是哪个版本

正文部分可以直接看 Skill 例子的正文，此处不展开赘述。

03
触发机制

Skills 的触发机制是整个系统中最为关键的环节，直接决定了 Skill 能否在恰当时机被激活。这一点甚至比 Skills 内容本身质量更为重要。

3.1 自动触发和手动触发

Skills 支持自动触发（语义匹配）和手动触发两种机制：

自动触发：靠的是 **description 字段**。当用户的意图和 Skill 描述语义匹配时，Agent 会自己判断"这个任务我有现成的专业流程可以用"，然后主动加载对应的 Skill。整个过程用户无感，就像一个老员工听到需求就自动翻出了对应的 SOP。

手动触发：用户通过斜杠命令（如 **/skill-name**）显式调用。这适合用户明确知道自己要用哪个 Skill 的场景，相当于直接点名"用这套流程来干活"。

两种机制互补：自动触发降低使用门槛，让 Skill 对新用户"隐形"生效；手动触发给老用户精确控制权，想用哪个你说了算。

3.2 description 书写规范

语义匹配机制完全依赖于 description 字段，description 是 Skill 能否被正确触发的核心。

Claude Code 推荐的书写公式为：**功能定义 + 触发场景 + 核心能力**。

写好它需要遵循几个原则：

- 同时回答 WHAT 和 WHEN。不能只说"处理文档"这么含糊，要明确写出这个 Skill 做什么、在什么场景下该被调用。
- 枚举具体的触发词：Agent 做语义匹配时，关键词越具体命中率越高。在 description 中显式列出用户可能使用的关键词——不只是术语，还包括口语化的说法。"创建实验"、"新建"、"做个实验"、"建个AB"——这些都要写进去。
- 用第三人称。description 会被注入系统提示词，所以要写成客观描述而非对话语气。写 "Generates API documentation from source code"，而不是 "I can help you generate docs"或"You can use this to..."。
- 划定排除边界（可选）：在描述中明确地标注出不适用的场景，降低误触发概率。

04
作用域与优先级

Skill 的作用域决定了它在哪些场景下可用。主流 Agent 产品通常支持两级作用域：

位置	生效范围	适用场景
企业配置中心	全员生效	强制执行的企业级开发规范与安全策略
用户主目录下全局配置	个人所有项目	通用工具、个人偏好、跨项目能力
项目根目录或 `.skills/` 目录	仅当前项目	项目特定工作流、团队约定
Plugin 内置资源	Plugin 启动时	社区共享的能力包、特定框架的专用指令集

当多个 Skill 同时存在时，可能出现触发冲突——两个 Skill 的 description 都匹配用户输入。优先级规则通常遵循以下层次：

企业策略 > 个人配置 > 项目配置 > Plugin 内置

05
Skill 最佳实践

5.1 Skill 正文即路由器：编排而非堆砌

核心理念：

- SKILL.md 的正文应该是"**路由器**"而非"知识仓库"，它的职责是分发任务到正确的模块，而非包含所有相关信息和业务细节；

- SKILL.md 应控制在 **500行** 以内，500行文本等于2000～3000 token，是单个 Skill 激活后比较合理的上下文开销。
- 在 SKILL.md 中引用辅助文件时，应当建立一份明确的契约：**触发的时机+资源位置+预期的产出**，不能只给出路径。

最佳实践：SKILL.md 只保留路由表和全局规则，业务细节下沉到模块文件。

```
# ✅ 最佳实践：SKILL.md 是路由器
## 意图路由表
| 场景示例              | 路由模块    | 加载文件                          |
|-----------------|-----------|-------------------------------|
| "创建实验"          | creator   | modules/creator/creator.md    |
| "实验XX调流量"       | modifier  | modules/modifier/modifier.md |
## 全局安全红线
1. 禁止调用万能工具
2. 禁止编造数据
3. 写模块限定接口
```

5.2 知识分层策略：何时拆分、如何组织

什么时候该拆文件？ 一条经验法则：当一个文件超过 300 行，或某个 Step 的规则超过 100 行时，就是拆分信号。文件太长不光浪费 token，还会让 Agent 的注意力在大段文本中迷失。

如何组织知识分层？根据使用频率对知识进行分层组织。 越频繁用到的知识，离入口越近；越偶尔查阅的知识，越往深处放。这样 Agent 每次只加载当前步骤真正需要的内容，不会把上下文窗口塞满无关信息。

实践中可以用一棵简单的决策树来判断：



实践：trade-ab-skill 的分层

```
trade-ab-skill/
├── SKILL.md                    ← 意图路由表、全局安全红线（每次激活必读）
├── modules/creator/creator.md            ← 创建流程 Step 编排、scenarioId
├── modules/creator/collect-phase.md      ← 参数填充的 11 项执行清单（仅 Ste
├── modules/creator/validate-phase.md     ← 校验规则（仅 Step 3 才读）
├── modules/creator/tools.md              ← MCP 工具接口列表（调接口时按需查阅
└── modules/creator/safety.md             ← 安全约束详细规则（validate 阶段按
```

5.3 安全实践 – Tool 的权限设计与工具隔离

这是工程级 Skill 最关键的安全实践之一。 当 Skill 涉及多个模块、调用多个 MCP 接口时，必须实施模块级的工具隔离——每个模块只能调用其白名单中的接口，遵循权限最小化原则。

设计原则：

1. **白名单制**：每个模块的 tools.md 明确列出可用接口，白名单外一律禁止；
2. **危险接口显式禁用**：万能工具（如直接 HTTP 调用）全局禁止；
3. **工具隔离**：不同模块使用不同的接口集合，防止误调用。

最佳实践：

- Claude Code 的 allowedTools 配置，控制 Skill 执行时可调用的工具，支持 Bash 的前缀匹配控制；
- trade-ab-skill 中的隔离实践：实验创建接口仅在 creator 白名单中；modifier 只能用实验修改接口；审批/发布接口在所有模块中均禁止调用；modifier 模块明确标注易混淆接口的禁止规则。

5.4 脚本增强：扩展 Skill 能力边界

核心理念：将确定性计算逻辑封装为脚本，由 Agent 调用执行而非自行推导。脚本是 Skill 突破 LLM 能力边界的利器。

什么时候需要写成脚本?

如果这件事让 LLM 做有概率出错，但让脚本做能 100% 确定性完成，那就该封装成脚本。

如果发现自己在 SKILL.md 中编写公式让 Agent 运行计算，该逻辑也应该被挪到脚本中。

场景	纯指令的局限	脚本的优势
配置文件读写	LLM 可能写入格式错误的 JSON	脚本保证格式正确，原子写入
环境检测	LLM 无法可靠检测系统状态	脚本直接查询，返回结构化结果
日志采集	LLM 不应直接处理网络请求	脚本封装 HTTP 调用，异常自处理

复杂计算	LLM 算术不可靠	脚本精确计算

脚本的设计遵循四个原则：

1. 自愈性——脚本内部处理所有异常，始终正常退出，绝不阻断 Skill 主流程
2. 结构化输出——统一输出 JSON，方便 Agent 解析和流转
3. 幂等性——多次执行结果一致，预检脚本只追加缺失项，不覆盖已有配置
4. 安全边界——只操作指定文件，不触碰其他系统资源

最佳实践：

trade-ab-skill 就内置了两个关键脚本：MCP预检脚本负责在 Skill 启动前自动检测 MCP 依赖是否就绪；日志采集脚本负责采集实验操作日志。Agent 只需要调用脚本、读取返回的 JSON，不用自己去"猜"环境状态。

5.5 参数传递与动态注入

Skill 不仅是静态指令，更支持运行时参数传递和上下文预注入。在执行过程中，需要在多个阶段之间传递参数。良好的参数传递机制应满足：

- **显式**：参数来源和去向清晰可追溯
- **可校验**：每个阶段有门卡检查参数完整性
- **防丢失**：关键参数在快照中持久化

最佳实践：trade-ab-skill 的参数传递模型：采用快照（Snapshot）机制作为跨阶段参数传递的载体。每个阶段将产出写入快照，下一阶段从快照读取。阶段门卡确保参数完整性。

动态注入：执行中的一些参数可以由用户直接提供，也可以在执行过程中动态获取：

- `businessName`：通过业务信息查询接口按当前用户工号动态查询
- `scenarioId`：根据用户输入的业务关键词，从对照表动态匹配
- `metricBindingBases`：固定使用指定模板展开的指标数组

最佳实践：trade-ab-skill 的用户偏好持久化：成功操作后，将关键参数写入 `user-prefs.json`，下次执行时自动注入：

```
{
  "defaultScenarioId": "<场景ID>",
  "defaultMetricTemplateId": "<模板ID>",
```

```
  "recentExperimentIds": ["<实验ID1>", "<实验ID2>"],
  "lastUsed": "2026-06-17",
  "usageCount": 2
}
```

5.6 测试与迭代

测试维度

Claude Code 官方推荐了三类核心测试方法：

- **触发测试**：测试 description 是否能被正确匹配，准备 10 个左右的自然语言变体去触发 Skill，检查是否都能正确激活，同时验证不相关的输入是否会误触发。
- **功能走查**：用自然语言驱动完整流程，检查每个阶段的输出是否符合预期。重点关注路由是否准确分发、渐进加载是否按时序工作、红线规则是否被遵守、异常场景是否正确熔断。别只跑 happy path——故意输入边界值、模拟工具不可用、尝试让 Agent 调用禁止接口，才是真正暴露问题的场景。
- **性能对比**：针对同一任务，分别用"无 Skill"和"有 Skill"两种方式各跑 5 次，对比 Token 用量和完成质量。

如何高效迭代？

- 与传统软件的 bug 修复逻辑一致：发现问题 -> 定位原因 -> 修复文档 -> 验证效果
- 较为高效的一种迭代方法是**观测驱动迭代**——通过日志埋点收集每次执行的状态结果（成功/失败/取消）、耗时、调用的工具列表，用数据定位薄弱环节。

最佳实践：

trade-ab-skill 的日志采集机制，采用了两阶段 traceId 配对追踪，保障了日志埋点的采集。

06
用 skill-creator 从零创建一个 Skill

skill-creator 是一个用来创建和打包 Skill 的辅助工具，本身也是一个 Skill。可以协助你从零搭建一个 Skill。

Step 0：前置准备

确保本地已安装以下工具：

- Git（已配置 SSH Key，可访问公司内部代码仓库）
- 一个可以进行对话的 AI 助手，可以使用 Qoder 等支持文件操作的工具

Step 1：初始化目录结构

告诉 AI：

> *「帮我用 skill-creator 生成一个名为 `order-query` 的 Skill」*

AI 会自动运行初始化脚本，生成标准目录：

```
order-query/
├── SKILL.md            ← 核心文件（必须）
├── scripts/            ← 可执行脚本（可选）
└── references/         ← 参考文档（可选）
```

Step 2：编写 SKILL.md

告诉 AI：

> *「帮我编写 order-query 的详细内容，这个 Skill 的功能是 xxx，主要用于 xxx 场景，触发词包括 xxx、xxx。」*

AI 会帮你生成包含 YAML frontmatter 和正文的完整 SKILL.md。写作要点：description 要包含所有可能的触发词；正文只写领域专有知识，不写常识；复杂内容拆到 `references/` 目录，SKILL.md 中用相对路径引用；控制在 500 行以内。

Step 3：添加参考文档（可选）

如果有复杂的操作流程或参考资料：

告诉 AI：

> *「把 xxx 的详细流程整理成 `references/workflow.md`，并在 SKILL.md 中加上引用。」*

Step 4：打包验证

告诉 AI：

> *「用 skill-creator 打包验证 `order-query`，检查格式是否合规。」*

打包脚本会自动校验：frontmatter 是否完整、`name` 和 `description` 是否存在、目录中是否有非法文件。有问题 AI 会直接告诉你哪里不对并帮你修复。

Step 5：推送到远端仓库

告诉 AI：

> *「帮我把 `order-query` 初始化为 git 仓库，关联远端地址，提交所有文件并推送到 main 分支。」*

Step 6：后续迭代更新

修改 Skill 内容后，告诉 AI：

> *「帮我提交 `order-query` 的最新改动，commit 信息是 xxx，然后推送。」*

整个流程可以浓缩为一张图：

```
下载 skill-creator
    ↓
告诉 AI：初始化目录
    ↓
告诉 AI：编写 SKILL.md + references
    ↓
告诉 AI：打包验证
    ↓
告诉 AI：git 提交并推送
    ↓
告诉 AI：迭代修改并推送

```

是不是很简单？从头到尾你没有手动执行过任何命令。你只负责描述意图，AI 负责执行，这就是

参考文档

[1] Agent Skills 官方文档：https://agentskills.io
[2]《Agent Skills 橙皮书：给AI装技能的完全指南》
[3]《Claude Code 实战 Harness 工程之道》
[4]《Harness 工程：从上下文管理到 Agent 系统构建》

**注：本文以集团内部 trade-ab-skill 项目为真实案例，所有代码示例均来自该项目实际运行的 Skill 文件。*



欢迎留言一起参与讨论~

阅读 9455

留言 6

写留言

五毛 四川 8月18日 1

最近在实战skill，在自己仓库迭代了几个，很有帮助，从文中也学到了标准目录结构，这个很有帮助

pandaman 北京 8月18日

@元宝文案提取 提取核心内容



Jh 天津 8月18日

牛哇



. 中国香港 8月17日

@元宝文案提取 提取文章内容



zxf945 浙江 8月17日

@元宝文案提取 总结下



毕竟是王哈哈 北京 8月17日

怎么自动化迭代skill

首评